1. Design a Student class with fields for name, roll number, and marks in 3 subjects. Create objects for 5 students and display the one with the highest total marks.

2. Create a Rectangle class with length and width. Write a program to calculate area and perimeter for multiple rectangle objects and find the one with the largest area.

3. Design a BankAccount class with account number, holder name, and balance. Implement deposit and withdrawal, ensuring withdrawal doesn't exceed the balance.

4. Create an Employee class with name, ID, and salary. Store multiple employee objects in an array and sort them by salary.

5. Design a Book class with title, author, price, and stock count. Simulate selling books and update stock accordingly.

6. Create a Car class with brand, model, and fuel level. Implement a drive(distance) method that reduces fuel based on distance travelled.

7. Design a Point class representing 2D coordinates. Write a method to calculate the distance between two Point objects.

8. Design a Product class with name, price, and quantity. Calculate the total inventory value across an array of product objects.

9. Create a Person class with name and age. Write a method to compare two Person objects and determine who is older.

10. Write a Calculator class with methods add, subtract, multiply, and divide, handling division by zero gracefully.

11. Design a Temperature class with a method to convert Celsius to Fahrenheit and vice versa.

12. Create a StringProcessor class with methods to reverse a string, count vowels, and check if it's a palindrome.

13. Design a Matrix class with a method to add two matrices of the same dimensions.

14. Write a Shape class (generic) with a method calculateArea() that behaves differently based on internal type fields (e.g., "circle" or "square").

15. Create an Inventory class with a method to search for a product by name and return its details.

16. Design a TimeConverter class with a method to convert total seconds into hours, minutes, and seconds.

17. Create a PrimeChecker class with a method that checks if a given number is prime.

18. Design a Library class with methods issueBook() and returnBook() that update book availability status.

19. Write a PayrollSystem class with a method to calculate net salary after deducting tax based on income slabs.

20. Create a Calculator class with an overloaded add() method that works for two integers, three integers, and two doubles.

21. Design a Printer class with overloaded print() methods to display an integer, a string, and an array of integers.

22. Write an AreaCalculator class with overloaded calculateArea() methods for a circle, rectangle, and triangle.

23. Create a Volume class with overloaded volume() methods for a cube, cuboid, and sphere.

24. Design a Greeting class with overloaded greet() methods: no arguments (default greeting), one argument (name), and two arguments (name and time of day).

25. Create a Converter class with overloaded convert() methods for converting int-to-string, double-to-string, and boolean-to-string.

26. Write a Search class with overloaded search() methods to find an integer in an array and a string in an array of strings.

27. Design a Discount class with overloaded applyDiscount() methods based on flat amount and based on percentage.

28. Create a Shape class with overloaded draw() methods that accept different numbers of parameters representing dimensions.

29. Write a Validator class with overloaded isValid() methods to validate an email string and a phone number (long).

30. Design a Student class with three constructors: default, one with name only, and one with name, roll number, and marks.

31. Create a Rectangle class with a constructor that takes no arguments (creates a unit square), one argument (creates a square), and two arguments (length and width).

32. Write a Book class with overloaded constructors: one accepting only the title, and another accepting title, author, and price.

33. Design an Employee class with constructors for (name), (name, salary), and (name, salary, department).

34. Create a Point class with a default constructor (origin) and a parameterized constructor for custom coordinates.

35. Design a Circle class with constructors accepting radius, or diameter with a boolean flag indicating which is provided.

36. Write a Vehicle class with constructors for different combinations of brand, model, and year.

37. Create a Fraction class with a constructor for a whole number and another for numerator/denominator pairs.

38. Design a Color class with constructors accepting RGB values, a hex string, or a named color string.

39. Write an Account class with constructors for creating a new account (auto-generated ID) and loading an existing account (given ID and balance).

40. Create a Counter class with a static variable that tracks the number of objects created so far.

41. Design an Employee class with a static method to calculate bonus based on a static company-wide bonus percentage.

42. Write a MathUtils class with static methods for factorial, GCD, and LCM (no object instantiation required).

43. Create a BankAccount class where the interest rate is static (shared across all accounts) but balance is instance-specific.

44. Design a Student class with a static variable for school name (common to all students) and instance variables for individual details.

45. Write a program demonstrating a static block that initializes a static configuration value when the class is first loaded.

46. Create an IDGenerator class using a static variable and static method to generate unique, auto-incrementing IDs for objects.

47. Design a Circle class with a static constant for PI and an instance method that uses it to calculate area.

48. Write a Logger class with a static method log(message) used across multiple unrelated classes without creating a Logger object.

49. Create a Product class with a static method to calculate total revenue across all product objects created, using a static list/array.

50. Design a Student class with a copy constructor that creates a new object as a duplicate of an existing student.

51. Create a Rectangle class with a copy constructor, and demonstrate that modifying the copy doesn't affect the original.

52. Write a Point class with a copy constructor and a method to verify two point objects have the same coordinates but are different objects.

53. Design an Employee class with a copy constructor used to create a duplicate employee record before giving a raise (to compare before/after).

54. Create a Book class with a copy constructor and demonstrate deep copying when the class contains an array of reviews.

55. Write a Matrix class with a copy constructor that performs a deep copy of the 2D array data.

56. Design a LinkedListNode-like Node class with a copy constructor that duplicates the node's data (not its reference links).

57. Create a Person class containing an Address object as a field; implement a copy constructor demonstrating deep vs shallow copy.

58. Write a ShoppingCart class with a copy constructor that duplicates the cart's item list independently of the original.

59. Design a Team class with a copy constructor used to clone a team roster before making experimental changes to player positions.

60. Design a Student class with overloaded constructors, a copy constructor, and a static variable tracking total students created.

61. Create a BankAccount class using constructor overloading (new account vs. existing account), static interest rate, and a copy constructor for creating account snapshots.

62. Write an Employee management system class with overloaded methods for salary calculation, overloaded constructors, and a static counter for total employees.

63. Design a Shape hierarchy simulated within one class using overloaded area() methods, overloaded constructors for different shapes, and a static shape counter.

64. Create a Library class with overloaded addBook() methods, a copy constructor for book records, and a static variable for total books in the library.

65. Design a Product class with constructor overloading, a copy constructor for creating discounted duplicates, and a static method to compute average price across all products.

66. Write a Game Character class with overloaded constructors (default vs. custom stats), a copy constructor for cloning characters, and a static variable tracking total characters spawned.

67. Design a Vehicle class combining constructor overloading (brand-only vs. full details), a copy constructor, overloaded display Info() methods, and a static registry counting all vehicles manufactured.

68. Create a Box class with length, breadth, and height. Write a method isLargerThan(Box b) that accepts a Box object and returns true if the calling object has greater volume. Compare three boxes and identify the largest.